

Applied Machine Learning

Multiclass linear classifiers



UNIVERSITY OF
GOTHENBURG

CHALMERS

Richard Johansson

`richajo@chalmers.se`

two-class (binary) linear classifiers

- a **linear classifier** is a classifier that is defined in terms of a scoring function like this

$$\text{score} = w \cdot x$$

- this is a **binary** (2-class) classifier:
 - return the first class if the score > 0
 - ...otherwise the second class
- how can we deal with non-binary (**multi-class**) problems when using linear classifiers?

approaches to multiclass classification

- **idea 1:** break down the complex problem into simpler problems, train a binary classifier for each separately
- **idea 2:** modify the learning algorithm so that it can handle the multiclass case directly

idea 1: reduction from multiclass to binary

- **one-versus-rest** (“long jump”):
 - for each class c , make a binary classifier to distinguish c from all other classes
 - so if there are n classes, there are n classifiers
 - at test time, we select the class giving the highest score
- **one-versus-one** (“football league”):
 - for each pair of classes c_1 and c_2 , make a classifier to distinguish c_1 from c_2
 - if there are n classes, there are $\frac{n \cdot (n-1)}{2}$ classifiers
 - at test time, we select the class that has most “wins”

example

- assume we're training a classifier of fruits and we have the classes apple, orange, mango
- in one-vs-rest, we train the following three classifiers:
 - apple vs orange+mango
 - orange vs apple+mango
 - mango vs apple+orange
- in one-vs-one, we train the following three:
 - apple vs orange
 - apple vs mango
 - orange vs mango

example (continued)

- we train classifiers to distinguish between apple, orange, and mango, using one-vs-rest
 - so we get w_{apple} , w_{orange} , w_{mango}
- for some instance x , the respective scores are

$[-1, 2.2, 1.5]$

so our guess is orange

in scikit-learn

- scikit-learn includes implementations of both of the methods we have discussed:
 - `OneVsRestClassifier`
 - `OneVsOneClassifier`
- however, the built-in algorithms (e.g. Perceptron, LogisticRegression) will do this automatically for you
 - they use one-versus-rest

idea 2: multiclass learning algorithms

- is it good to separate the multiclass task into smaller tasks that are trained independently?
 - maybe training should be similar to testing?
- let's make a model where one-vs-rest is used while training
 - we'll see how this can be done for logistic regression

binary LR: reminders

- the **logistic** or **sigmoid** function:

$$P(\text{positive output}|x) = \frac{1}{1 + e^{-\text{score}}}$$

```
def sigmoid(score):  
    return 1 / (1 + np.exp(-score))
```

- when training, we minimize the **log loss** (binary cross-entropy)

$$\text{Loss}(w, x, y) = \log(1 + \exp(-y \cdot (w \cdot x)))$$

multiclass LR using the softmax

- the **softmax** function is used in multiclass LR instead of the logistic:

$$P(y_i | \mathbf{x}) = \frac{e^{\text{score}_i}}{\sum_k e^{\text{score}_k}}$$

```
def softmax(scores):  
    expscores = np.exp(scores)  
    return expscores / sum(expscores)
```

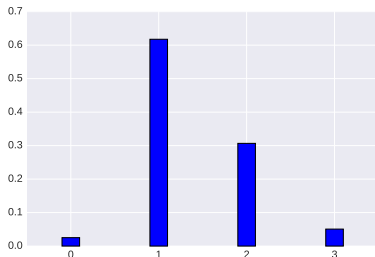
[exercise: make softmax numerically stable]

softmax example

```
def softmax(scores):  
    expscores = np.exp(scores)  
    return expscores / sum(expscores)
```

```
scores = [-1, 2.2, 1.5, -0.3]  
print(softmax(scores))
```

```
array([ 0.02517067,  0.61750026,  0.30664156,  0.05068751])
```



cross-entropy loss

- when training, the softmax probabilities lead to the **cross-entropy loss** instead of the log loss

$$\text{Loss}_{CE}(w, x_i, y_i) = -\log P(y_i|x_i) = -\log \frac{e^{\text{score}_i}}{\sum_k e^{\text{score}_k}}$$

- just like the log-loss:
 - high probability for the correct label $y_i \Rightarrow$ low loss
 - low probability for $y_i \Rightarrow$ high loss

multiclass LR in scikit-learn

- `LogisticRegression(multi_class='multinomial')`
- (otherwise, separate classifiers are trained independently)

conclusion

linear classifiers can be generalized to **more than two classes**

solution 1: **reduction to binary classifiers**

solution 2: **multiclass by design**

